

文件操作

复习

1. STL概述
2. STL基本组成
3. string容器
4. vector容器
5. stack容器
6. queue容器
7. deque容器
8. list容器
9. set/multiset容器
10. map/multimap容器
11. STL常用算法

了解

熟悉

重点

重点

重点

重点

重点

重点

重点

重点

重点

目录

- | | |
|----------------|----|
| 1. 数据流的概念 | 了解 |
| 2. 文件处理简介 | 了解 |
| 3. 文件的输入与输出 | 重点 |
| 4. 文件写入案例 | 熟悉 |
| 5. 文件读取案例 | 熟悉 |
| 6. 二进制文件读写 | 重点 |
| 7. fstream文件操作 | 重点 |
| 8. 文件定位和大小 | 重点 |

数据流的概念

数据流（data stream）是一组有序，有起点和终点的字节的数据序列。包括输入流和输出流。

c++通过一种称为流（stream）的机制提供了更为精良的输入和输出方法。流是一种灵活且面向对象的I/O方法。

根据操作对象不同分为文件流、字符串流、控制台流。

控制台流

C++输入输出操作分别是由istream(输入流)和ostream(输出流)这两个类提供的，为了允许双向的输入 / 输出，由istream和ostream派生出了iostream类。

类的继承关系如下图：



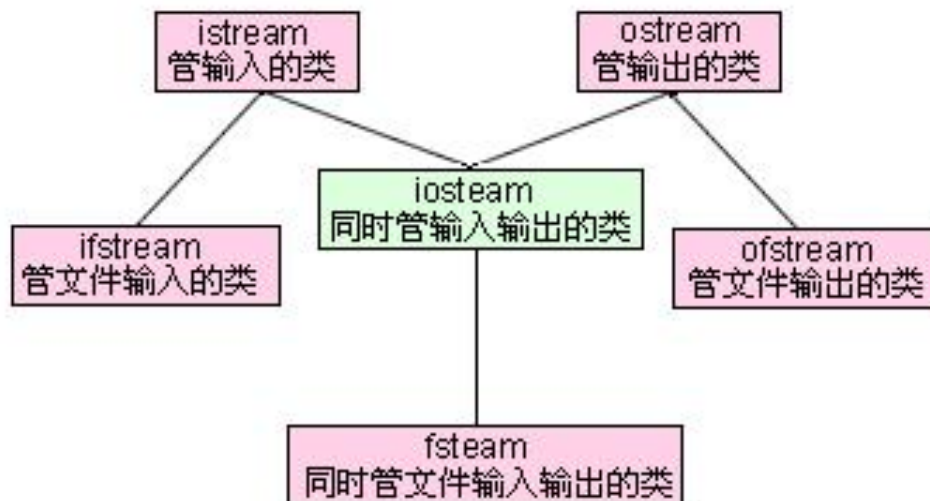
是一种多继承的关系，在程序应用可以只包含头文件iostream。

函数	功能	应用
cin	输入(standard input)的istream类对象	从设备读入数据
cout	输出(standard output)的ostream类对象	向设备输出或者写数据
cerr	标准错误(standard error)的ostream类对象	屏幕设备写数据

上面的<<和>>操作符，是因为iostream.h头文件中，ostream类对应每个基本数据类型都有其友元函数对左移操作符进行了友元函数的重载。

文件流

文件流的输入输出类在fstream头文件被定义，和控制台流继承关系为：



文件流

由于文件设备不像显示器屏幕和键盘那样标准的默认设备，所以我们定义一个流对象。

ofstream类的默认构造函数原形为：

```
ofstream::ofstream(constchar *filename,int mode = ios::out, int penprot = filebuf::openprot);
```

参数	值来源
filename	打开的文件名
mode	打开文件的模式(ios:out)
prot	打开文件属性 (filebuf::openprot)

文件流

mode属性表

类型	解释
ios::app	追加方式打开文件
ios::ate	文件打开后定位到文件尾，app就包含有此属性
ios::binary	以二进制方式打开文件，缺省的方式是文本方式
ios::in	文件以输入方式打开 (ifstream)
ios::out	文件以输出方式打开 (ofstream)
ios::nocreate	不建立文件，所以文件不存在时打开失败
ios::noreplace	不覆盖文件，打开文件时如果文件存在失败

文件流

打开文件属性值

类型	解释
0	普通文件，打开访问
1	只读文件
2	隐含文件
4	系统文件

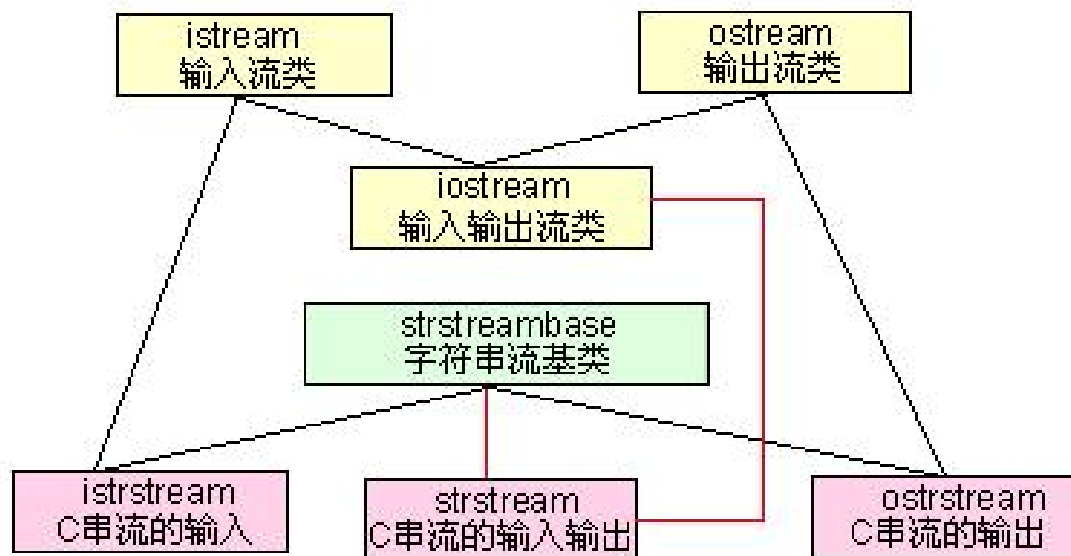
“或”或者“+”把以上属性连接起来，如3或1|2就是以只读和隐含属性打开文件。文件使用完后可以使用close成员函数关闭文件。

ios::app为追加模式，在使用追加模式的时候同时进行文件状态的判断是一个比较好的习惯。

字符串流

字符串流就是能够控制字符串类型对象进行输入输出的类，C++不光可以支持C++风格的字符串流控制，还可以支持C风格的字符串流控制。

继承关系如下图：



istrstream

C风格的串流的输入操作，字符串数组作为输入。

ostrstream

C风格的串流的输出操作，字符串数组作为输出。

stringstream

支持C风格的串流的输入输出操作。

文件处理简介

c++的文件处理也是看为一个对象,使用文件流的类,使用头文件 `<fstream>`,处理的时候有**文本文件和二进制文件之分**,主要的区别就是存储的形式。

C++ 通过以下几个类支持文件的输入输出：

ofstream: 写操作（输出）的文件类 (由ostream引申而来)

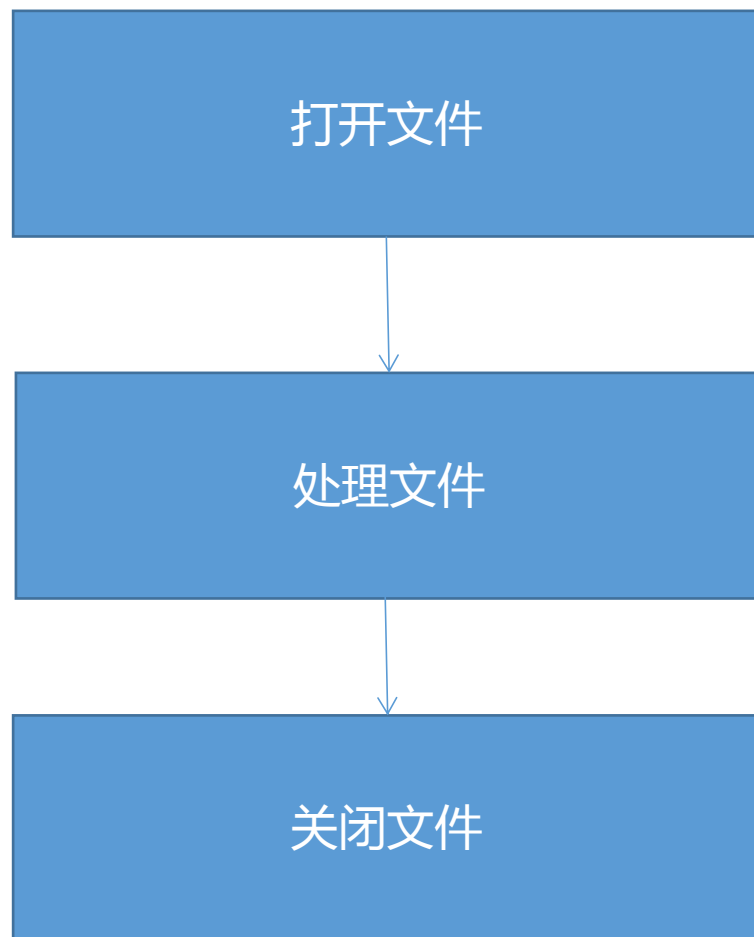
ifstream: 读操作（输入）的文件类 (由istream引申而来)

fstream: 可同时读写操作的文件类 (由iostream引申而来)

要在 C++ 中进行文件处理，必须在 C++ 源代码文件中包含头文件 `<iostream>` 和 `<fstream>`。

文件处理简介

文件处理基本流程：



文件的输入与输出

打开文件

在从文件读取信息或者向文件写入信息之前，必须先打开文件。

ofstream 和 fstream 对象都可以用来打开文件进行写操作，如果只需要打开文件进行读操作，则使用 ifstream 对象。

下面是 `open()` 函数的标准语法，`open()` 函数是 fstream、ifstream 和 ofstream 对象的一个成员。

```
void open(const char *filename, ios::openmode mode);
```

`open()` 成员函数的第一参数指定要打开的文件的名称和位置，第二个参数定义文件被打开的模式。

文件的输入与输出

1. 打开文件

在从文件读取信息或者向文件写入信息之前，必须先打开文件。

ofstream 和 fstream 对象都可以用来打开文件进行写操作，如果只需要打开文件进行读操作，则使用 ifstream 对象。

下面是 `open()` 函数的标准语法，`open()` 函数是 fstream、ifstream 和 ofstream 对象的一个成员。

```
void open(const char *filename, ios::openmode mode);
```

`open()` 成员函数的第一参数指定要打开的文件的名称和位置，第二个参数定义文件被打开的
模式。

文件的输入与输出

模式标志	描述
<code>ios::app</code>	追加模式。所有写入都追加到文件末尾。
<code>ios::ate</code>	文件打开后定位到文件末尾。
<code>ios::in</code>	打开文件用于读取。
<code>ios::out</code>	打开文件用于写入。
<code>ios::trunc</code>	如果该文件已经存在，其内容将在打开文件之前被截断，即把文件长度设为0。

可以把以上两种或两种以上的模式结合使用。例如，如果想要以写入模式打开文件，并希望截断文件，以防文件已存在，那么可以使用下面的语法：

```
ofstream outf;  
outf.open("file.txt", ios::out | ios::trunc );
```

如果想要打开一个文件用于读写，可以使用下面的语法：

```
fstream file;  
file.open("file.txt", ios::out | ios::in );
```

文件的输入与输出

ofstream, ifstream 和 fstream 所有这些类的成员函数 open 都包含了一个默认打开文件的方式，这三个类的默认方式各不相同：

类	参数的默认方式
ofstream	ios::out ios::trunc
ifstream	ios::in
fstream	ios::in ios::out

文件的输入与输出

2.写入与读取

写入文件

在 C++ 编程中，可以使用流插入运算符（ << ）向文件写入信息，就像使用该运算符输出信息到屏幕上一样。唯一不同的是，在这里使用的是 ofstream 或 fstream 对象，而不是 cout 对象。

读取文件

在 C++ 编程中，可以使用流提取运算符（ >> ）从文件读取信息，就像使用该运算符从键盘输入信息一样。唯一不同的是，在这里使用的是 ifstream 或 fstream 对象，而不是 cin 对象。

文件的输入与输出

3.关闭文件

当 C++ 程序终止时，它会**自动关闭刷新所有流**，释放所有分配的内存，并关闭所有打开的文件。但写程序时应该养成一个好习惯，在程序终止前**关闭所有打开的文件**。

下面是 close() 函数的标准语法，close() 函数是 fstream、ifstream 和 ofstream 对象的一个成员。

```
void close();
```

文件写入案例

```
#include <iostream>
#include <fstream> //1.头文件
using namespace std;
int main() {
    //2.创建流对象
    ofstream outf;
    //3.打开文件
    outf.open("file.txt"); //outf.open("file.txt",ios::out);
    //4.写入内容
    outf << " 山村咏怀" << endl;
    outf << " 邵雍" << endl;
    outf << "一去二三里，烟村四五家。" << endl;
    outf << "亭台六七座，八九十枝花。" << endl;
    //5.关闭文件
    outf.close();

    return 0;
}
```



每次运行都会覆盖原来的内容

文件写入案例

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {

    ofstream outf;
    outf.open("file.txt",ios::app);
    outf << "    山村咏怀" << endl;
    outf << "    邵雍" << endl;
    outf << "一去二三里，烟村四五家。" << endl;
    outf << "亭台六七座，八九十枝花。" << endl;
    outf.close();

    return 0;
}
```



每次运行都不会覆盖
原来的内容

文件写入案例 总结

- 1.包含头文件
- 2.创建对象
- 3.打开文件
- 4.写入内容
- 5.关闭文件

文件读取案例

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    //char cf[100]; //字符数组
    string cf; //字符串
    ifstream infile;
    infile.open("file.txt",ios::in); //inf.open("file.txt");
    infile >> cf;
    infile.close();
    cout << cf;

    return 0;
}
```



能读取全部内容吗？

文件读取案例

在c++中使用ifstream读文件的时候会以空格为分隔符，遇到空格就不读取了。这时候可以调用get函数来读取内容。

```
#include<iostream>
#include<fstream> //文件流的头文件
using namespace std;
int main()
{
    char a;
    ifstream infile;    //定义ifstream类(输入文件流类)对象infile
    infile.open("file.txt");//打开文件 使文件流与c++.txt文件建立关联
    while (!infile.eof())
    {
        infile.get(a); //依次获取文件中每个字符 并输出
        cout << a;
    }
    infile.close();      //关闭文件 使文件流与c++.txt文件断开关联
}
```

文件读取案例

```
#include<iostream>
#include<fstream>
#include<string>
using namespace std;
int main()
{
    ifstream ifs;
    ifs.open("file.txt", ios::in);
    //判断是否打开
    if (!ifs.is_open())
    {
        cout << "文件打开失败！" << endl;
        return 0;
    }
    //读数据
    //第一种：效率低，不建议使用
    /*char c;
    while ((c = ifs.get()) != EOF)
    {
        cout << c;
    }*/
```

```
//第二种
/*char buf[1024] = {0};
while (ifs >> buf)
{
    cout << buf << endl;
}*/
//第三种
/*char buf[1024] = {0};
while (ifs.getline(buf, sizeof(buf)))
{
    cout << buf << endl;
}*/
//第四种
string buf;
while (getline(ifs,buf)) //getline函数需要#include <string>
{
    cout<<buf<<endl;
}
ifs.close();
return 0;
}
```


文件读取案例 总结

- 1.包含头文件
- 2.创建对象
- 3.打开文件
- 4.读取内容
- 5.关闭文件

二进制文件读写

写文件

二进制写文件主要利用流对象调用成员函数 `write ( )`

函数原型：`ostream& write(const char*buffer,int len);`

参数解释：字符指针buffer指向内存中的一段存储空间，len是读写的字节数

二进制文件读写 案例

```
#include <iostream>
#include <fstream>
using namespace std;
class Student {
public:
    Student() {}
    Student(string name,int age) :name(name), age(age) {}
protected:
    string name;
    int age;
};
int main() {

    ofstream bof;
    Student s1("张三", 100);
    bof.open("student.txt", ios::out|ios::binary);
    bof.write((const char*) & s1, sizeof(Student));
    bof.close();
    return 0;
}
```

二进制文件读写

读文件

二进制读文件主要利用流对象调用成员函数`read`

函数原型：`istream& read( char*buffer,int len);`

参数解释：字符指针`buffer`指向内存中的一段存储空间，`len`是读写的字节数

二进制文件读写 案例

```
ifstream bif;  
bif.open("student.txt", ios::in | ios::binary);  
if (!bif.is_open()) {  
    cout << "文件打开失败" << endl;  
    return 0;  
}  
Student s2;  
bif.read((char*)&s2, sizeof(Student));  
cout << s2.name << "  " << s2.age << endl;  
bif.close();
```

fstream文件操作

ifstream支持>>操作，ofstream支持<<操作，fstream同时支持>>和<<操作。

fstream文件操作

打开文件 `fstream`可以在声明流对象时传入文件名打开文件，也可以使用`open()`函数打开文件。

关闭文件 文件打开后必须关闭，`fstream`提供`close()`函数关闭文件。

打开文件

使用构造函数声明对象时打开文件，示例：

```
fstream file(filename, ios::in | ios::out);
```

使用`open()`函数打开文件，函数原型：

```
void open(const char* filename,int mode,int access);
```

fstream文件操作

如果只传入文件名，系统会自动根据文件类型选择默认的打开模式。
打开文件模式mode如图：

mode	打开文件模式	说明
	ios::app	以追加的方式打开文件
	ios::ate	文件打开后定位到文件尾，ios::app包含此属性
	ios::binary	以二进制方式打开文件，默认是文本文件
	ios::in	以输入方式打开，读出文件内容
	ios::out	以输出方式打开，写入文件内容
	ios::nocreate	以不创建文件的方式打开文件，如果文件不存在，打开失败
	ios::noreplace	以不覆盖的方式打开文件，如果文件存在，打开失败
	ios::trunc	如果文件存在，清空文件内容

fstream文件操作

模式和属性可以单独使用给，也可以混合使用。混合使用时，用逻辑连接符或 | 连接。

ios::out模式默认会清空，即ios::out|ios::trunc和ios::out打开文件时都会清空文件。

如果不想清空文件，那么设置读写模式为ios::out|ios::app，以这种模式打开文件后，数据会以追加的方式写入到文件。

fstream提供的读写操作有：<<、>>、read()、write()、put()、get()、getline()。根据文件类型可分为文本文件和二进制文件，根据读写方式可分为逐行读写和整块读写。通常，文本文件使用逐行读写的方式，二进制文件使用整块读写的方式。

fstream文件操作

读写文件

fstream提供的读写操作有：<<、>>、read()、write()、put()、get()、getline()。根据文件类型可分为文本文件和二进制文件，根据读写方式可分为逐行读写和整块读写。通常，文本文件使用逐行读写的方式，二进制文件使用整块读写的方式。

fstream文件操作

1. 文本文件读写

逐行写入文本文件可以使用操作符<<。

逐行读取文本文件时可以使用getline()函数。

fstream文件操作

2. 二进制文件的读写

二进制文件通常整块读取或写入，当然也可以读写单个字符，用到的函数包括：`put()`、`get()`、`read()`、`write()`。

通常使用`write()`、`put()`函数写入二进制文件。使用`read()`、`get()`读取二进制文件。

fstream文件操作 案例

```
#include <iostream>
#include <fstream> //1.包含头文件
#include <string>
using namespace std;
int main() {
    string str;
    fstream iofile; //2.创建对象
    iofile.open("file.txt", ios::out | ios::in | ios::trunc); //3.打开文件

    iofile << "这里是写入内容测试" << endl; //4.写入数据
    iofile << "this is test" << endl;
    iofile.close(); //5.关闭文件
    cout << "写入完毕" << endl;
    iofile.open("file.txt", ios::out | ios::in);
    while (getline(iofile, str)) //循环读取
    {
        cout << str << endl;
    }
    iofile.close();
    cout << "读取完毕" << endl;

    return 0;
}
```

文件定位和大小

C++的文件定位分为读位置和写位置的定位，对应的成员函数分别为seekg()和seekp()。seekg()函数设置读位置，seekp()设置写位置。函数原型如下：

`istream& seekg(streamoff offset, seek_dir origin);`

`ostream& seekp(streamoff offset, seek_dir origin);`

offset表示偏移量，seek_dir表示移动的基准位置，取值如下：

`ios::beg` 文件开头

`ios::cur` 文件当前位置

`ios::end` 文件结尾

示例：

`inFile.seekg(2, ios::beg);` // 把文件读指针从开始位置向后移动2个字节

`outFile.seekp(2, ios::cur);` // 把文件写指针从当前位置向后移动2个字节

获取文件大小可以使用seekg()和tellg()或者seekp()和tellp()函数结合使用的方式获取文件大小。

示例：

`inFile.seekg(0, ios::end);` // 读文件指针移动到文件末尾

`streampos ipos = inFile.tellg();` // 返回当前指针的位置，也就是文件的大小，单位是字节

文件定位和大小 案例

如何求出一个文件的字节大小？

文件定位和大小 案例代码

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    long start, end;
    ifstream ifile;
    ifile.open("test.txt", ios::in | ios::binary);
    start = ifile.tellg();
    ifile.seekg(0, ios::end);
    end = ifile.tellg();
    ifile.close();

    cout << (end - start) << " bytes.\n"; //文件字节大小
    return 0;
}
```


知识点总结

- | | |
|----------------|----|
| 1. 数据流的概念 | 了解 |
| 2. 文件处理简介 | 了解 |
| 3. 文件的输入与输出 | 重点 |
| 4. 文件写入案例 | 熟悉 |
| 5. 文件读取案例 | 熟悉 |
| 6. 二进制文件读写 | 重点 |
| 7. fstream文件操作 | 重点 |
| 8. 文件定位和大小 | 重点 |

作业

1. 把自己喜欢的古诗或者句子，写入file1.txt文件里面，然后再读取出来显示到控制台。

下节课预告

综合案例6